

DATA STRUCTURES & ALGORITHMS

Complete Course Notes | Implementation in Java

Theory + Definitions + Memory Diagrams + Practical Java Code + Real-World Applications | 5 Units

SYLLABUS COVERED (click-to-topic guide inside)

Unit 1 DS concepts, Data vs Information, ADT, memory representation, Arrays, Singly/Circular/Doubly Linked Lists, Polynomial via LL	Unit 2 Stacks (ADT + impl), multiple stacks, infix→postfix, postfix evaluation, recursion, Queues, Circular, Deque, Priority Queue, simulation	Unit 3 Tree terms, BST, AVL, Heap, B/B+/B*/Red-Black tree, forest, multi-way tree, comparisons	Unit 4 Graph types, matrix/list representation, BFS, DFS, MST (Kruskal, Prim), Dijkstra, algorithm comparison
Unit 5 Bubble, Selection, Insertion, Shell, Merge, Quick, Heap, Radix sorts; Sequential & Binary search; Hashing & Indexing; OS-DBMS case study	Style Boxed definitions, memory diagrams, code boxes, comparison tables, exercises	Language Note Java everywhere. Pointer-style memory topics shown with C-style diagrams where the syllabus demands it.	How to use Read Unit 1→2→3→4→5 in order; run every code file in any JDK (e.g. javac Name.java; java Name)

Prepared as a self-study / exam-prep guide. All programs were written to be compilable with plain JDK (no external libraries).
Topics that the classic C syllabus expresses with pointers are explained side-by-side with the Java reference equivalent.

TABLE OF CONTENTS

Introduction to Data Structures, Arrays & Linked List	Data vs manipulation
Stacks & Queues	Stack ADT queue · simple
Trees	Definition comparison
Graphs	Classification
Sorting, Searching, Hashing & Case Study	8 sorting

Companion way to study: implement each heading's code in your editor, run it, and redraw the diagram on paper. Practise the boxed "Exercise" lines at the end of every unit.

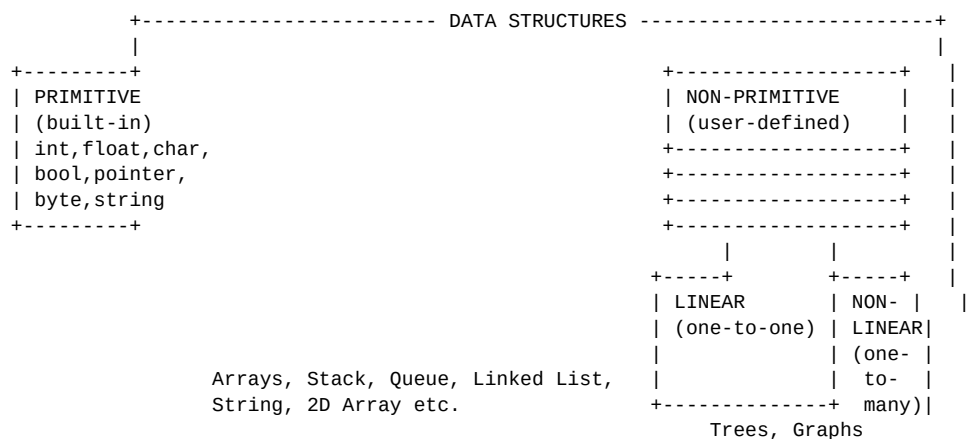
1.1 Data vs Information — Concepts

Data are raw, unprocessed facts and figures collected from observations, measurements or transactions (e.g., a number "42", a word "Ahmedabad", marks = 87). Data alone has no meaning. When data is processed, organised and given context, it becomes Information (e.g., "87 marks means pass with distinction").

Information is processed/meaningful data. Example: raw numbers 78, 82, 91 are data; "average = 83.6, so the class performed well" is information.

A data structure is a systematic way of organising and storing data in a computer so that it can be used efficiently. It defines the logical arrangement of data plus the set of operations to access and manipulate it. Example: a phone book (array/list of contacts), a metro route map (graph).

Classification of Data Structures



Class	Examples	Characteristic	Real-world use
Primitive	int, float, char, boolean, double, long	Directly operated by machine hardware	Storing a single value (age, price)
Linear (non-primitive)	Array, Stack, Queue, Linked List	Elements arranged in a sequence, one after another (one-to-one)	Queue in billing counter
Non-linear (non-primitive)	Tree, Graph	Elements connected hierarchically or arbitrarily (one-to-many/many-to-many)	File-system folders (tree), road network (graph)
Static vs Dynamic	Array (static), Linked List (dynamic)	Static: fixed size; Dynamic: size grows/shrinks at runtime	Array for fixed table, LL for dynamic cart

1.2 Abstract Data Type (ADT)

An ADT is a mathematical model of a data type defined by the behaviour (set of operations) from the user's point of view, hiding the internal implementation details. A name + domain of values + operations = ADT.

ADT = Interface Contract + Hidden Implementation

```

+-----+
|          STACK ADT (behaviour only)          |
| Values   : homogeneous items, LIFO order      |
| Operations: push(x), pop(), peek(), isEmpty(), |
|             size()                           |
| (Implementation may use an ARRAY or a LINKED LIST - the |
|   user only sees the same behaviour.)         |
+-----+
| Real world: UNDO stack in a text editor, browser BACK |
|   button, call stack of function calls, "recent files". |
+-----+

```

Example in Java: Stack interface (ADT) implemented two ways**StackADT.java - interface hides how data is stored**

```

interface StackADT {
    void push(int x);  int pop();    int peek();
    boolean isEmpty(); boolean isFull();  int size();
}
// 1) array implementation      2) linked-list implementation
class ArrayStack implements StackADT {    class LinkedStack implements StackADT {
    private int[] a; int top = -1;          private Node head; ...
    public void push(int x){a[++top]=x;}    // same 5 methods, different internals
}

```

1.3 Memory Representation & Implementation Aspects**TWO PHYSICAL STORAGE METHODS**

1) CONTIGUOUS (sequential) storage: data occupies consecutive memory addresses (arrays, string). 2) LINKED (non-contiguous) storage: each element stores its data plus the address of the next element (linked list); nodes may sit anywhere in RAM.

Contiguous vs Linked Memory Layout

CONTIGUOUS (int A[5] starting at address 1000, 4 bytes each):

```

+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+
addr=1000    1004    1008    1012    1016
address(A[2]) = base + index * size = 1000 + 2*4 = 1008

```

LINKED (3 Node objects scattered anywhere in RAM):

```

+-----+ +-----+ +-----+
| 10 | addr#2000 |-->| 20 | addr#4100 |-->| 30 | NULL |
+-----+ +-----+ +-----+
Node at #1000    Node at #2000    Node at #4100

```

Each node = data part + pointer(reference) part. Java stores the reference(32/64 bit address) just like a C pointer.

1.4 Operations on Data Structures & Cost Estimation**Common operations**

Traversal Visit every element once. Cost = $O(n)$	Search Find an element. Array: $O(n)$ linear / $O(\log n)$ if sorted.	Insertion Add at a position. Array worst $O(n)$ (shift), LL $O(1)$ at head.	Deletion Remove element. Array $O(n)$ (shift), LL $O(1)$ at head after search
Sorting Arrange elements. $O(n \log n)$ best for comparison sorts	Merging Combine two sorted lists -> one sorted list. $O(n)$	Access by index Array $O(1)$, linked list $O(n)$	Update Combine find + write; same cost as search

ASYMPTOTIC (BIG-OH) COST

Cost is measured as a function of input size n (worst case). Big-Oh gives the upper bound of growth: $O(1)$ constant (access `array[i]`), $O(\log n)$ (binary search), $O(n)$ (linear search), $O(n \log n)$ (merge sort), $O(n^2)$ (bubble sort), $O(2^n)$ (recursive Fibonacci). We ignore constants and lower-order terms.

Operations	Array	Singly Linked List	Remarks
Access index i	$O(1)$	$O(n)$	Array wins by random access
Insertion at beginning	$O(n)$ (shift all)	$O(1)$	LL wins
Insertion at end	$O(1)$ if free tail, else $O(n)$	$O(n)$ without tail pointer	Need size
Deletion	$O(n)$	$O(1)$ after finding node ($O(n)$ find)	LL wins for frequent inserts
Search	$O(n)$	$O(n)$	Both linear
Memory extra	None	$O(n)$ for link field	LL costs extra pointer space

1.5 Linear DS — Arrays

DEFINITION: ARRAY

An array is a finite, ordered collection of homogeneous (same-typed) elements stored in consecutive memory locations, referenced by an index (0-based in Java). Random access means any element is reachable in $O(1)$ time using `base + index * elementSize`.

1-D and 2-D Array Memory Mapping (address computation)

```
int[] marks = {78, 92, 60}; 1-D array layout (base 5000, int=4 bytes):
index:      0      1      2
            +-----+-----+-----+
            | 78   | 92   | 60   |
            +-----+-----+-----+
address : 5000   5004   5008      address = base + idx*4

2-D array int a[3][4] (3 rows, 4 columns) by ROW-MAJOR order (Java):
row0: a00 a01 a02 a03 | row1: a10 a11 a12 a13 | row2: a20 a21 a22 a23
address(a[i][j]) = base + (i * Ncols + j) * size      (Ncols = 4)
```

ArrayDemo.java - creation, traversal, search, 2-D

```
public class ArrayDemo {
    public static void main(String[] args) {
        int[] marks = new int[5];           // declare + allocate
        for (int i = 0; i < marks.length; i++) // traversal
            marks[i] = (i + 1) * 10;

        int max = marks[0], sum = 0;
        for (int m : marks) { sum += m; if (m > max) max = m; } // search+update
        System.out.println("Sum=" + sum + " Max=" + max);

        // 2-D array: represent a 3x4 marks table
        int[][] table = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
        for (int i = 0; i < table.length; i++) {
            for (int j = 0; j < table[i].length; j++)
                System.out.print(table[i][j] + " "); // row-major visit
            System.out.println();
        }
    }
}
```

Array Operation	Worst-case cost	Java code pattern
Access element i	$O(1)$	<code>arr[i]</code>
Traversal	$O(n)$	<code>for (int x : arr) {...}</code>

Array Operation	Worst-case cost	Java code pattern
Linear search	$O(n)$	for/foreach + if match
Binary search (sorted)	$O(\log n)$	Arrays.binarySearch(arr, key)
Insert at position (shift)	$O(n)$	System.arraycopy + arr[i]=v
Delete position (shift)	$O(n)$	System.arraycopy
Update	$O(1)$	arr[i] = newValue

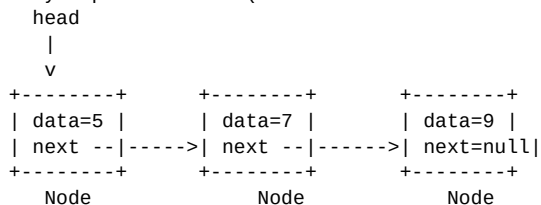
1.6 Linked List

DEFINITION: LINKED LIST

A linked list is a linear DS made of NODES. Each node = data field + a reference (pointer in C) to the next node. Unlike arrays, elements are NOT contiguous; the list can grow/shrink dynamically and insertions/deletions are cheap, but there is no random access.

Linked Representation of a List (data + link per node)

Memory representation (Java references shown as arrows):



In C the same diagram is drawn with pointers:

```
struct Node { int data; struct Node *next; };
Java's object reference next behaves exactly like a pointer.
```

Real-world uses of Linked List: browser back/forward history, image viewer next/previous (doubly), undo-redo in editors, music playlist, LRU cache, free-memory management in OS (free list), adjacency list of graphs, polynomial arithmetic.

Singly Linked List — Java implementation

SinglyLinkedList.java - add, delete, search, traverse

```

class Node { int data; Node next; Node(int d){ data=d; } }

class SinglyLinkedList {
    Node head;

    void insertAtHead(int d) {           // O(1)
        Node n = new Node(d);
        n.next = head;   head = n;
    }
    void insertAtTail(int d) {           // O(n)
        Node n = new Node(d);
        if (head == null) { head = n; return; }
        Node t = head;
        while (t.next != null) t = t.next;
        t.next = n;
    }
    void delete(int key) {               // O(n) : find + unlink
        if (head == null) return;
        if (head.data == key) { head = head.next; return; }
        Node t = head;
        while (t.next != null && t.next.data != key) t = t.next;
        if (t.next != null) t.next = t.next.next;
    }
    boolean search(int key) {           // O(n)
        for (Node t = head; t != null; t = t.next)
            if (t.data == key) return true;
        return false;
    }
    void traverse() {                   // O(n) : visit all
        for (Node t = head; t != null; t = t.next)
            System.out.print(t.data + " -> ");
        System.out.println("null");
    }
    public static void main(String[] a) {
        SinglyLinkedList l = new SinglyLinkedList();
        l.insertAtHead(9); l.insertAtTail(7);
        l.insertAtHead(5); l.traverse();    // 5 -> 9 -> 7 -> null
        l.delete(9);      l.traverse();    // 5 -> 7 -> null
        l.insertAtTail(4); l.traverse();    // 5 -> 7 -> 4 -> null
    }
}

```

Variants: Circular & Doubly Linked List**CIRCULAR LINKED LIST**

Last node's next points back to the first node (no null). Advantage: you can reach any node from any node; used in round-robin CPU scheduling, token-ring networks, Josephus problem, repeating music playlists.

DOUBLY LINKED LIST

Each node has two links: prev and next. Supports walking in both directions and O(1) deletion of a known node. Used in browser history, undo/redo stacks (java.util.LinkedList is a doubly-linked list).

```

DOUBLY list:  null<-[5]<=>[7]<=>[9]->null
CIRCULAR list: [5]->[7]->[9]
                ^       |
                +-----v (9.next = 5, no null)

CIRCULAR DOUBLY list: head[5]<=>[7]<=>[9]->back to head

```

Doubly Linked List - insert operation (Java)

```

class DNode { int data; DNode prev, next; DNode(int d){data=d;} }
// insert after node 'p':
DNode n = new DNode(data);
n.next = p.next; n.prev = p;
if (p.next != null) p.next.prev = n;
p.next = n;

```

1.7 Application of Linked List — Polynomial Manipulation**WHY A LINKED LIST?**

A polynomial like $5x^3 + 4x^2 - 3x + 7$ has a variable number of terms. Only the non-zero coefficients are stored: each node = coefficient + exponent + pointer to next term. This is memory-efficient for sparse polynomials (a linked node list).

$5x^3 + 4x^2 - 3x + 7$ is stored as:

```

+-----+ +-----+ +-----+ +-----+
| 5 | 3 | |-->| 4 | 2 | |-->| -3 | 1 | |-->| 7 | 0 | null
+-----+ +-----+ +-----+ +-----+
(coef,exp) (coef,exp) (coef,exp) (coef,exp)

```

Polynomial.java - addition of two sparse polynomials

```

class PNode { int coef, exp; PNode next;
    PNode(int c, int e){ coef=c; exp=e; } }

class Polynomial {
    PNode head = null;

    void insert(int c, int e) {          // keep nodes sorted by exponent (desc)
        PNode n = new PNode(c, e);
        if (head == null || head.exp < e) { n.next = head; head = n; return; }
        PNode t = head;
        while (t.next != null && t.next.exp > e) t = t.next;
        n.next = t.next; t.next = n;
    }

    Polynomial add(Polynomial q) {      // poly addition -> O(m + n)
        Polynomial r = new Polynomial();
        PNode a = head, b = q.head;
        while (a != null && b != null) {
            if (a.exp == b.exp) { r.insert(a.coef + b.coef, a.exp); a=a.next; b=b.next; }
            else if (a.exp > b.exp) { r.insert(a.coef, a.exp); a=a.next; }
            else { r.insert(b.coef, b.exp); b=b.next; }
        }
        while (a != null) { r.insert(a.coef, a.exp); a=a.next; }
        while (b != null) { r.insert(b.coef, b.exp); b=b.next; }
        return r;
    }

    void display() {
        for (PNode t = head; t != null; t = t.next)
            System.out.print(t.coef + "x^" + t.exp + " + ");
        System.out.println("0");
    }

    public static void main(String[] a) {
        Polynomial p = new Polynomial();          // 5x^3 + 4x^2 - 3x + 7
        p.insert(7,0); p.insert(-3,1); p.insert(4,2); p.insert(5,3);
        Polynomial q = new Polynomial();          // 2x^2 + 6x + 1
        q.insert(1,0); q.insert(6,1); q.insert(2,2);
        Polynomial r = p.add(q);
        p.display(); q.display(); r.display();    // 5x^3 + 6x^2 + 3x + 8
    }
}

```


Note: in the C syllabus the same concept uses `struct Node{ int coef; int exp; struct Node* next; }`; Java has no pointer syntax but the reference member `next` is equivalent. The algorithm is language-independent.

Real-world: polynomial lists are used in computer algebra systems (Mathematica, sympy), error-correcting codes (CRC/Cyclic Redundancy Check computation), digital filters, and Bézier curve math.

Exercise: 1) Trace `insertAtHead` for 4,6,8 on paper. 2) Implement polynomial multiplication using the same list. 3) Convert a singly list into a circular list and test round-robin print.

UNIT 2 - STACKS & QUEUES

2.1 Stacks as an ADT

DEFINITION: STACK

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle. Insertion (PUSH) and deletion (POP) happen only at one end called the TOP. The element added last is removed first.

Stack Working - LIFO, top = only open end

```

push(5)  push(9)  push(3)          pop()          top=3
+-----+
top=1    top=2    top=3          | 3 | <-- top (TOS)
+-----+ +-----+ +-----+   +-----+
| 5 | top | 9 | top | 3 | top |   | 9 |
+-----+ +-----+ +-----+   +-----+
| 9 |    | 5 |    | 9 |    |   | 5 |
+-----+ +-----+ +-----+   +-----+
| 5 |    | 5 |    | ...   pop() returns 9 (removed)
+-----+ +-----+ +-----+   remaining: 5 waiting
bottom is never touched (only top end is opened)

```

Operation	Result	Worst cost
push(x)	Adds x on top	O(1)
pop()	Removes & returns top	O(1)
peek()/top()	Returns top w/o removing	O(1)
isEmpty()	true if no element	O(1)
isFull()	true if array stack full	O(1)
size()	Count of elements	O(1)

2.2 Different Implementations of Stack

A) Array (static / bounded size)**StackArray.java - array implementation**

```

class StackArray {
    private int[] a; private int top = -1;
    StackArray(int cap){ a = new int[cap]; }

    void push(int x) {
        if (top == a.length-1) throw new RuntimeException("Stack OVERFLOW (isFull)");
        a[++top] = x;           // increment then store
    }
    int pop() {
        if (top == -1) throw new RuntimeException("Stack UNDERFLOW (isEmpty)");
        return a[top--];       // fetch then decrement
    }
    int peek(){ return a[top]; }
    boolean isEmpty(){ return top == -1; }
    boolean isFull(){ return top == a.length-1; }
}

```

B) Linked List (unbounded / dynamic)

StackLL.java - linked implementation (no isFull needed)

```

class StackLL {
    private Node top;    // reuse : class Node { int data; Node next; }
    void push(int x){ Node n = new Node(x); n.next = top; top = n; }
    int pop(){ int v = top.data; top = top.next; return v; }
    int peek(){ return top.data; }
    boolean isEmpty(){ return top == null; }
}

```

Multiple Stacks — two stacks sharing one array

```

array size = N=10 ; stack1 grows from left, stack2 grows from right
[ | |9| | | ... | | |7| | ]
top1=2                top2=7
push1: a[++top1];    push2: a[--top2];
FULL when top1 + 1 == top2    (stacks meet)

```

TwoStacks.java - memory-efficient multiple stacks

```

class TwoStacks {
    int[] a; int top1 = -1, top2;
    TwoStacks(int n){ a = new int[n]; top2 = n; }
    void push1(int x){ if(top1+1 < top2) a[++top1]=x; else overfull(); }
    void push2(int x){ if(top2-1 > top1) a[--top2]=x; else overfull(); }
    int pop1(){ return a[top1--]; } int pop2(){ return a[top2++]; }
}
// Generalisation: k stacks in one array (divide array into k fixed sections).

```

2.3 Application 1 — Infix to Postfix Conversion**DEFINITIONS**

INFIX: operator between operands: A + B (needs brackets & precedence). POSTFIX (reverse Polish): operator after operands: A B + — evaluated left-to-right without brackets. PREFIX: operator before: + A B.

Rule-based algorithm (using a stack of operators)**Pseudocode - Shunting-Yard (Dijkstra) for infix to postfix**

```

for each token t in the infix expression (L to R):
    if t is an OPERAND      -> append t to output
    if t is '('             -> push t
    if t is ')'             -> pop & append operators until '(' ; discard '('
    if t is an OPERATOR     -> while stack top is operator with
                              (precedence(t) <= precedence(top))
                              and t is left-associative: pop to output
                              then push t
after all tokens: pop any remaining operators to output

```

InfixToPostfix.java - stack based conversion

```

import java.util.*;
public class InfixToPostfix {
    static int prec(char c){
        switch(c){ case '+': case '-': return 1;
                   case '*': case '/': case '%': return 2;
                   case '^': return 3; default: return 0; } }
    static String convert(String infix){
        StringBuilder out = new StringBuilder();
        Deque<Character> st = new ArrayDeque<>(); // stack
        for(char t : infix.toCharArray()){
            if(Character.isLetterOrDigit(t)) out.append(t);
            else if(t=='(') st.push(t);
            else if(t==')'){ while(!st.isEmpty() && st.peek()!='(')
                             out.append(st.pop()); st.pop(); }
            else { // operator
                while(!st.isEmpty() && st.peek()!='(' &&
                       prec(t) <= prec(st.peek()))
                    out.append(st.pop());
                st.push(t);
            }
        }
        while(!st.isEmpty()) out.append(st.pop());
        return out.toString();
    }
    public static void main(String[] a){
        System.out.println(convert("A+B*C")); // ABC*+
        System.out.println(convert("(A+B)*C")); // AB+C*
        System.out.println(convert("A+B*(C-D)")); // ABCD-*+
    }
}

```

Infix	Postfix	Prefix	Steps (stack trace for A+B*C)
A+B*C	A B C * +	+ A * B C	out=ABC, st=[*,+] -> pop *, pop +
(A+B)*C	A B + C *	* + A B C	on) : pop + ; then out, then *
A*B+C	A B * C +	+ * A B C	since * higher, push before +
A^B^C (right assoc)	A B C ^ ^	^ A ^ B C	^ handled right-associative (else order breaks)

2.4 Application 2 — Evaluation of Postfix Expression

IDEA

Scan the postfix string left to right. Push operands. When an operator appears, pop two operands (b = pop, a = pop), compute (a op b), push the result back. At the end the stack holds the answer.

PostfixEval.java - single pass, stack machine

```

import java.util.*;
public class PostfixEval {
    static int eval(String p){
        Deque<Integer> st = new ArrayDeque<>();
        for(char t : p.toCharArray()){
            if(Character.isDigit(t)) st.push(t-'0');
            else { int b = st.pop(), a = st.pop();      // ORDER! b popped 2nd
                switch(t){
                    case '+': st.push(a+b); break;
                    case '-': st.push(a-b); break;
                    case '*': st.push(a*b); break;
                    case '/': st.push(a/b); break;
                }
            }
        }
        return st.pop();
    }
    public static void main(String[] a){
        System.out.println(eval("23*5+"));    // (2*3)+5 = 11
        System.out.println(eval("56+7*"));    // (5+6)*7 = 77
        System.out.println(eval("234*+"));    // 2+3*4 = 14
    }
}

```

Trace of "23*5+" :

```

token 2 : push => [2]
token 3 : push => [2,3]
token * : pop 3, pop 2 => 6 push => [6]
token 5 : push => [6,5]
token + : pop 5, pop 6 => 11 push => [11]    ANSWER = 11

```

This is exactly how a JVM/CPU evaluates expressions - a stack machine!
C compilers also translate infix to postfix for evaluation.

Real world: expression calculators, compiler code generation, JVM bytecode, RPN calculators (HP calculators), spreadsheet formula parsers, undo/redo (store past states), recursion call stack.

2.5 Application 3 — Recursion

DEFINITION: RECURSION

A function that calls itself directly or indirectly. Each call pushes an activation record (arguments, local vars, return address) onto the RUN-TIME STACK; each return pops it. Without a base case the stack overflows (StackOverflowError).

Recursion.java - factorial, fib, Tower of Hanoi

```

public class Recursion {
    static long fact(int n){                // factorial (linear recursion)
        if (n <= 1) return 1L;              // base case stops the recursion
        return n * fact(n-1);              // recursive step
    }
    static int fib(int n){                  // tree recursion (slow  $O(2^n)$ )
        if (n <= 1) return n;
        return fib(n-1) + fib(n-2);
    }
    // Tower of Hanoi : move n disks from A -> C using B as temporary
    static void hanoi(int n, char from, char aux, char to){
        if (n == 1) { System.out.println(from + " -> " + to); return; }
        hanoi(n-1, from, to, aux);          // move n-1 to aux
        System.out.println(from + " -> " + to); // move biggest disk
        hanoi(n-1, aux, from, to);          // move n-1 to to
    }
    public static void main(String[] a){
        System.out.println(fact(5));         // 120
        hanoi(3, 'A', 'B', 'C');
    }
}

```

fact(4) stack frames (each call = one frame):
 fact(4)-->4*fact(3)-->3*fact(2)-->2*fact(1)=2
 frames: [4][3][2][1] then unwind: 2,6,24

A	called frames (push)	returned values (pop)
+4+	+4+ +4+ +4+	+4+
+3+ ->	+3+ +3+ -> ...	<- +3+
	+2+ +2+	+2+ each -*
	+1+	+1+

2.6 Queues as an ADT

DEFINITION: QUEUE

A queue is a linear data structure following FIFO (First In, First Out). Insertion occurs at REAR (enqueue), deletion at FRONT (dequeue). Like a railway ticket counter - the first person in line is served first.

QueueArray.java - array implementation with modular indexing

```

class QueueArray {
    private int[] a; private int front=0, rear=-1, size=0;
    QueueArray(int cap){ a = new int[cap]; }
    void enqueue(int x){                // insert at rear
        if (size == a.length) throw new RuntimeException("OVERFLOW");
        rear = (rear + 1) % a.length; a[rear] = x; size++;
    }
    int dequeue(){                      // remove from front
        if (size == 0) throw new RuntimeException("UNDERFLOW");
        int v = a[front]; front = (front + 1) % a.length; size--;
        return v;
    }
    int peek(){ return a[front]; }
    boolean isEmpty(){ return size == 0; }
}

```

Circular Queue (ring buffer) — removal of empty-front problem

Circular Queue - reuse empty slots

linear queue wastes space: after dequeue from front, empty cells at the front cannot be reused. CIRCULAR queue wraps rear to front:
 $\text{rear} = (\text{rear} + 1) \% \text{size}$; $\text{front} = (\text{front} + 1) \% \text{size}$

```

[ ][ ][40][50][ ] enqueue(60):
      ^f      ^r      rear=(2+1)%5=3 -> put at 3
[60][ ][40][50][ ] front=2 rear=3, full: size==cap
      ^      ^f      ^r

```

USED in: CPU process queue (ring buffers), audio/video buffers, printer spool, keyboard circular buffer, network packet queues.

2.7 Deque, Priority Queue & Queue Simulation**DEQUE (DOUBLE-ENDED QUEUE)**

Insert and delete allowed at BOTH ends. Two varieties: input-restricted (insert only at rear) and output-restricted (delete only at front). Java's ArrayDeque implements it. Used in A-Steal scheduling, palindrome checking, browser undo/redo across two ends.

PRIORITY QUEUE

The element with Highest Priority leaves first (not arrival time). Implemented with a heap. Java: PriorityQueue<E>. Used in printing jobs (VIP first), OS process scheduling, Dijkstra, Huffman coding, emergency rooms (critical patients first).

JavaCollectionsDemo.java - Deque & Priority Queue from java.util

```

import java.util.*;

public class JavaCollectionsDemo {
    public static void main(String[] a){
        // Deque via ArrayDeque (double-ended)
        ArrayDeque<Integer> dq = new ArrayDeque<>();
        dq.addLast(10); dq.addLast(20); dq.addFirst(5);
        System.out.println(dq.pollFirst() + " " + dq.pollLast()); // 5 20

        // PriorityQueue : min-heap by default (smallest leaves first)
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.add(30); pq.add(10); pq.add(20);
        System.out.println(pq.poll()); // 10 (highest priority = smallest)

        // max-priority: custom comparator
        PriorityQueue<Integer> maxpq =
            new PriorityQueue<>(Comparator.reverseOrder());
        maxpq.add(30); maxpq.add(10);
        System.out.println(maxpq.poll()); // 30
    }
}

```

Queue Simulation

Simulation = imitate a real system with its events and measure behaviour. Example: a bank with 3 counters. We maintain one arrival queue + events (customer arrival time, service time). Randomly generate 1000 customers, use discrete-event simulation, count average waiting time & queue length. The queue DS itself is the heart of the simulator.

QueueSim.java - simulation of arrival/service queue

```
// Discrete event simulation skeleton for a single-server counter
import java.util.*;
public class QueueSim {
    public static void main(String[] args){
        Random r = new Random(1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        int clock = 0, waitSum = 0, n = 0;
        while (n < 200) {
            if (r.nextInt(10) < 4) {                // 40% chance arrival
                queue.addLast(clock); n++;
            }
            if (!queue.isEmpty() && r.nextInt(10) < 6){ // 60% chance service
                int t = queue.pollFirst();
                waitSum += clock - t;                // time spent waiting
            }
            clock++;
        }
        System.out.println("avg waiting time ~ " + (waitSum / (double) n));
    }
}
```

Feature	Stack	Queue	Deque	Priority Queue
Ordering	LIFO	FIFO	FIFO + at both ends	By priority (not time)
Insert	push (top)	enqueue (rear)	addFirst/addLast	add, heap-sift up
Remove	pop (top)	dequeue (front)	removeFirst/Last	poll (root)
Java class	Deque+push, or Stack	ArrayDeque/List	ArrayDeque	PriorityQueue
Real use	Undo, call stack	Print spool, scheduling	Palindrome, browser fwd/back	OS priority, Dijkstra

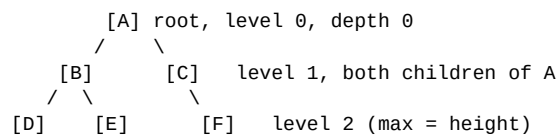
Exercise: 1) Convert $A+(B*C-(D/E^F))*G$ to postfix on paper and verify with the code. 2) Trace postfix $8\ 2\ 3\ * +$ for value 14. 3) Implement a circular queue and print head/tail after each op.

UNIT 3 - TREES

3.1 Tree Definitions — Height, Depth, Order, Degree

DEFINITION: TREE

A tree is a non-linear, hierarchical data structure: a finite set of nodes with exactly one ROOT and every other node reachable by exactly one path (no cycles). Used for one-to-many relationships.

Tree Terminology Visual

Key terms:

Node : element + child references
 Root : topmost node, no parent (A)
 Leaf : node with no children (D,E,F)
 Internal : node with at least one child (A,B,C)
 Edge : link between parent and child
 Degree : number of children of a node (B has 2, C has 1)
 Order : maximum allowed degree of the whole tree
 Height : longest path from root to a leaf in edge-count = 2 here
 Depth : length of path from ROOT to the node (D depth = 2)
 Level : depth + 1 (root at level 1)
 Sibling : children of same parent (D,E)
 Subtree : a node with all its descendants (B,D,E is a subtree)

Concept	Meaning	Example (above)
Depth of node	Edges from root to node	depth(A)=0, depth(D)=2
Height of tree	Max depth over all nodes	height = 2
Height of node	Longest downward path to a leaf	height(B)=1
Degree of node	Count of its children	deg(A)=2, deg(F)=0
Order of tree	Maximum degree in tree	this tree order = 2

3.2 Binary Search Tree (BST)

DEFINITION: BST

A Binary Tree where every node satisfies: left subtree contains only smaller keys, right subtree contains only larger keys. No duplicates (usually). This ordering makes search/insert/delete $O(\log n)$ on a balanced BST, $O(n)$ worst (skewed).

BST.java - insert, delete, search, inorder (sorted output)

```

class BST {
    class Node { int key; Node left, right; Node(int k){key=k;} }
    Node root;

    Node insert(Node r, int k){                // O(height)
        if (r == null) return new Node(k);
        if (k < r.key)    r.left = insert(r.left, k);
        else if (k > r.key) r.right = insert(r.right, k);
        return r;
    }
    boolean search(Node r, int k){              // O(height)
        if (r == null) return false;
        if (r.key == k) return true;
        return k < r.key ? search(r.left, k) : search(r.right, k);
    }
    int minKey(Node r){ while(r.left != null) r = r.left; return r.key; }

    Node delete(Node r, int k){                // O(height)
        if (r == null) return null;
        if (k < r.key) r.left = delete(r.left, k);
        else if (k > r.key) r.right = delete(r.right, k);
        else {
            if (r.left == null) return r.right; // found it // 0/1 child
            if (r.right == null) return r.left;
            r.key = minKey(r.right);           // 2 children: take in-order
            r.right = delete(r.right, r.key);   // successor, then remove it
        }
        return r;
    }
    void inorder(Node r){ if(r != null){ inorder(r.left);
        System.out.print(r.key+" "); inorder(r.right); } }
    public static void main(String[] a){
        BST t = new BST();
        for (int x : new int[]{50,30,70,20,40,60,80}) t.root = t.insert(t.root, x);
        t.inorder(t.root);                      // 20 30 40 50 60 70 80
        t.root = t.delete(t.root, 50);          // delete root
        System.out.println(); t.inorder(t.root); // 20 30 40 60 70 80
        System.out.println(t.search(t.root, 60)); // true
    }
}

```

Traversal	Visit order	Output on the tree in 3.2	Use
Inorder (L-Rt-R)	left, node, right	20 30 40 50 60 70 80	Sorted order
Preorder (Rt-L-R)	node, left, right	50 30 20 40 70 60 80	Copy / serialize tree
Postorder (L-R-Rt)	left, right, node	20 40 30 60 80 70 50	Delete tree, expression eval
Level-order (BFS)	top to bottom, L to R	50 30 70 20 40 60 80	Shortest hop, printing layers

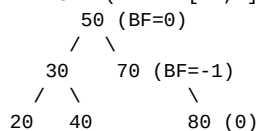
Real world of BST: CPU scheduling by priority search trees, spell-check dictionaries, autocomplete, symbol tables in compilers, relational DB indexes (variants), file folders.

3.3 AVL Tree (self-balancing BST)

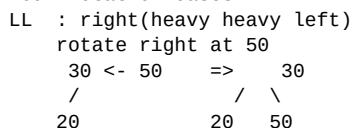
DEFINITION: AVL TREE

Adelson-Velskii & Landis tree = BST + balance condition: for every node, $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$. Balance factor $\text{BF} = h(L) - h(R)$. If a node's BF becomes +2 or -2 after insert/delete, we REBALANCE with rotations. Guarantees $O(\log n)$ height always.

BALANCED (BF in [-1,1])



four rotation cases:



Unbalanced insert cases & the single / double rotations:

LL -> single right_rotate | RR -> single left_rotate

LR -> left_rotate(child) then right_rotate (double)

RL -> right_rotate(child) then left_rotate (double)

AVL.java - insert with LL/RR/LR/RL rebalancing

```

class AVL {
    class Node { int k, h=1; Node l, r; Node(int x){k=x;} }
    Node root;
    int h(Node n){ return n==null?0:n.h; }
    int bf(Node n){ return h(n.l)-h(n.r); }
    Node rotR(Node y){ Node x=y.l; Node t=x.r; x.r=y; y.l=t;
        y.h=1+Math.max(h(y.l),h(y.r)); x.h=1+Math.max(h(x.l),h(x.r)); return x; }
    Node rotL(Node x){ Node y=x.r; Node t=y.l; y.l=x; x.r=t;
        x.h=1+Math.max(h(x.l),h(x.r)); y.h=1+Math.max(h(y.l),h(y.r)); return y; }
    Node insert(Node r, int k){
        if (r==null) return new Node(k);
        if (k<r.k) r.l =insert(r.l,k); else if(k>r.k) r.r=insert(r.r,k);
        else return r;
        r.h = 1+Math.max(h(r.l),h(r.r));
        int b = bf(r);
        if (b>1 && k<r.l.k) return rotR(r); // LL
        if (b<-1 && k>r.r.k) return rotL(r); // RR
        if (b>1 && k>r.l.k){ r.l=rotL(r.l); return rotR(r); } // LR
        if (b<-1 && k<r.r.k){ r.r=rotR(r.r); return rotL(r); } // RL
        return r;
    }
    public static void main(String[] a){
        AVL t = new AVL();
        for (int x : new int[]{10,20,30,40,50,25}) t.root=t.insert(t.root,x);
        // after 10,20,30 -> RR rotation; tree stays balanced height ~2
        System.out.println("height = " + t.h(t.root));
    }
}

```

Real world of AVL: in-memory sorts & indexes (java TreeSet/TreeMap use Red-Black, a similar balanced tree), language dictionaries, database main-memory caches.

3.4 Heap (binary tree in an array)

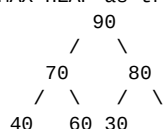
DEFINITION: HEAP

A COMPLETE Binary Tree (all levels full except possibly last, filled left-to-right) stored in an ARRAY with the heap property:

MAX-heap: parent $\geq$ children; MIN-heap: parent $\leq$ children. Root = max/min. Used for priority queues and heap sort.

Heapify is $O(\log n)$.

MAX-HEAP as tree



same heap as array (0-based):

```

index : 0  1  2  3  4  5
value : 90 70 80 40 60 30
parent(i) = (i-1)/2
left(i)   = 2i+1
right(i)  = 2i+2

```

insert: append at end, then siftUp

deleteMax: replace root with last element, then siftDown (heapify)

MaxHeap.java - sift-up insert, sift-down extractMax (extract prints sorted desc)

```

class MaxHeap {
    int[] a; int n=0;
    MaxHeap(int cap){ a = new int[cap]; }
    void swap(int i,int j){ int t=a[i]; a[i]=a[j]; a[j]=t; }
    void insert(int x){
        a[n++] = x;
        int i = n-1;
        while (i>0 && a[(i-1)/2] < a[i]) { swap(i,(i-1)/2); i=(i-1)/2; } // sift up
    }
    int extractMax(){
        // O(log n)
        int mx = a[0];
        a[0] = a[--n];
        int i = 0;
        while (true){
            // sift down
            int l=2*i+1, r=2*i+2, big=i;
            if (l<n && a[l]>a[big]) big=l;
            if (r<n && a[r]>a[big]) big=r;
            if (big==i) break;
            swap(i,big); i=big;
        }
        return mx;
    }
    public static void main(String[] a){
        MaxHeap h = new MaxHeap(20);
        for (int x : new int[]{40,70,90,30,60,80})
            h.insert(x);
        while (h.n>0) System.out.print(h.extractMax() + " "); // 90 80 70 60 40 30
    }
}

```

Real world of heaps: operating-system priority scheduling (kernel run-queue), Dijkstra shortest path (priority queue of vertices), Huffman coding, Java PriorityQueue, in-memory top-k queries ("top 10 trending"), median of streaming data.

3.5 Forest, Multi-Way Tree, B / B+ / B* & Red-Black Tree

FOREST

A forest is a collection (disjoint set) of trees. Removing the root of a tree gives a forest. Converting a forest to a binary tree: keep the first child as left link, next siblings as right links (left-child, right-sibling representation).

MULTI-WAY / M-ARY TREE

Each node can have MORE than two children (order m). Search nodes hold multiple keys. B-tree is the most famous member; generalized multi-way trees power disk indexes.

B-Tree of order 4 (each node can hold up to 3 keys, up to 4 children):

```

      [ 30 | 70 ]
      /   |   \
[10|20|25] [40|50] [80|90|95]

```

B+ tree = all keys/data go to LEAVES; internal nodes carry only "routing keys"; leaves are linked in sorted order -> fast range scans.

B* tree = B+ with higher minimum fill (2/3) -> fewer splits.

Red-Black tree = BST + colour rule: no two consecutive red nodes; root is black; every root-leaf path has same black height -> $O(\log n)$.

TreeCollections.java - Red-Black tree in java.util

```
// Java gives ready-made balanced trees for everyday use:
import java.util.*;
public class TreeCollections {
    public static void main(String[] a){
        TreeMap<Integer,String> tm = new TreeMap<>(); // Red-Black tree
        tm.put(3,"three"); tm.put(1,"one"); tm.put(2,"two");
        System.out.println(tm.keySet()); // [1, 2, 3]
        System.out.println(tm.ceilingKey(2)); // 2 (range queries)

        TreeSet<Integer> ts = new TreeSet<>();
        ts.add(50); ts.add(30); ts.add(70);
        System.out.println(ts); // [30, 50, 70]
    }
}
// No built-in B/B+ tree in Java standard lib; use external libraries
// (e.g. H2/MySQL internals) - B+ tree is a database index structure.
```

Tree	Order (children)	Balance	Used for
BST	2	Not guaranteed	In-memory sorted search
AVL	2	Strict ($ BF \leq 1$)	Few lookups, heavy inserts: DB memory
Red-Black	2	Relaxed (black height)	Java TreeMap/TreeSet, C++ std::map, Linux kernel
B-tree	m (many)	All leaves same depth	File systems (NTFS), DB indexes
B+ tree	m (many)	All leaves same depth	MySQL/PostgreSQL indexes (fast range scan)
Heap	2 but complete	Heap property only	Priority queues, heap sort
B-tree family	Where data lives	Leaf links	Typical use
B-tree	In every node (internal too)	No	General disk index, files
B+ tree	Only in leaves (internal = keys only)	Yes (linked leaves)	Relational DB indexes
B* tree	Only in leaves	Yes	Denser nodes (2/3 fill), fewer splits
Red-Black	Memory tree, 2 children	No	In-program ordered collections

Real world of trees: MS Windows folder tree + Java TreeMap (red-black), MySQL InnoDB index (B+ tree), ext4 filesystem (HTree = hash B-tree), Google BigTable/MongoDB (B-tree), priority schedule (heap), org-charts & family trees, game tree (chess AI), XML/JSON DOM (tree).

Exercise: 1) Insert 10,20,30,40,50 into AVL and draw rotations. 2) Build a min-heap on paper from 40,10,30,5,8 (draw the array). 3) Why does B+ tree beat BST on a 10^9 -row table? 4) Write iterative inorder for BST.

UNIT 4 - GRAPHS

4.1 Introduction & Classification of Graphs

DEFINITION: GRAPH

A graph $G = (V, E)$ is a pair of a set of VERTICES (V) and a set of EDGES (E) connecting pairs of vertices. Graphs model arbitrary many-to-many relations: cities+roads, users+friends, web pages+links.

UNDIRECTED (edges = two-way)

```

1 ---- 2
|  \   |
(3)---(4)

```

weights
1,2: 5

DIRECTED / DIGRAPH (arrows)

(1) ----> (2)

v edges with weight:
(3) ----> (4)
 $G = (V, E)$, $E = \text{directed pairs } (u, v)$

weighted

unweighted

connected vs disconnected

cyclic vs acyclic

(a tree = connected acyclic graph)

sparse (few edges)

dense (many edges)

Type	Edge meaning	Example
Undirected	Relationship both ways: $(u, v) = (v, u)$	Facebook friends, roads
Directed (digraph)	One-way: $u \rightarrow v \neq v \rightarrow u$	Twitter follows, web links
Weighted	Edge carries a cost/distance	Road kilometres
Unweighted	All edges equal (weight 1)	Friend graph
Cyclic / Acyclic	Contains / no cycle	Cycle=deadlock/loop
Connected	Path between every pair	Communication network
Dense / Sparse	Near full / mostly empty edges	Airport mesh / social graph

4.2 Representation — Adjacency Matrix & Adjacency List

GRAPH: (0)-(1)-(2) and edge 0-2 (3 not used)

0 1 2

0 [0 1 1]

1 [1 0 1]

2 [1 1 0]

Adjacency MATRIX: $A[u][v]=1$ if edge $u-v$;
space $O(V^2)$, edge-existence $O(1)$.

Adjacency LIST (ArrayLists per vertex):

0 -> [1, 2]

1 -> [0, 2]

2 -> [0, 1]

space $O(V + E)$ | better for sparse graphs

GraphRep.java - build matrix + list from same graph

```

import java.util.*;
public class GraphRep {
    // --- adjacency LIST : array of ArrayList<Integer> ---
    static ArrayList<Integer>[] list;
    // --- adjacency MATRIX : boolean[][] ---
    static boolean[][] mat;

    static void init(int n){
        list = new ArrayList[n];
        for (int i=0;i<n;i++) list[i] = new ArrayList<>();
        mat = new boolean[n][n];
    }
    static void addU(int u, int v){          // undirected
        list[u].add(v); list[v].add(u);
        mat[u][v] = mat[v][u] = true;
    }
    static void addD(int u, int v){          // directed
        list[u].add(v); mat[u][v] = true;
    }
    public static void main(String[] a){
        init(3);
        addU(0,1); addU(1,2); addU(0,2);
        for (int i=0;i<3;i++) System.out.println(i + " -> " + list[i]);
        // 0 -> [1, 2]   1 -> [0, 2]   2 -> [1, 0]
        System.out.println(mat[0][2]);    // true
    }
}

```

	Adjacency Matrix	Adjacency List
Space	$O(V^2)$ always	$O(V + E)$
Edge (u,v) check	$O(1)$	$O(\deg(v))$
List all neighbours of v	$O(V)$	$O(\deg(v))$
Batch-write edge	$O(1)$	$O(1)$ (amortised)
Best for	Dense graphs, small n	Sparse graphs (most real graphs)

4.3 Graph Traversal — BFS and DFS

BFS (BREADTH FIRST SEARCH)

Starts at a source vertex, visits all neighbours first (level by level) using a QUEUE. Finds the SHORTEST hop-path in unweighted graphs. $O(V+E)$. Used in peer-to-peer networks, GPS levels of friends tracking, web crawlers, bipartite checks.

DFS (DEPTH FIRST SEARCH)

Explores as far as possible along a branch before backtracking, using a STACK (recursion uses the call stack). $O(V+E)$. Used in maze solving, cycle detection, topological sort, counting connected components, Sudoku/AI backtracking.

Traversals.java - BFS via queue, DFS via recursion

```

import java.util.*;
public class Traversals {
    static ArrayList<Integer>[] g;
    static boolean[] vis;

    static void bfs(int start){
        ArrayDeque<Integer> q = new ArrayDeque<>();    // FIFO queue
        q.add(start); vis[start] = true;
        while(!q.isEmpty()){
            int u = q.poll();
            System.out.print(u + " ");
            for (int v : g[u])
                if (!vis[v]) { vis[v]=true; q.add(v); }
        }
    }
    static void dfs(int u){                                // recursion = stack
        vis[u] = true;
        System.out.print(u + " ");
        for (int v : g[u]) if (!vis[v]) dfs(v);
    }
    public static void main(String[] a){
        int n = 5;
        g = new ArrayList[n];
        for (int i=0;i<n;i++) g[i] = new ArrayList<>();
        ArrayList<int[]> edges = new ArrayList<>();
        Collections.addAll(edges, new int[]{0,1},new int[]{0,2},
            new int[]{1,3},new int[]{2,3},new int[]{3,4});
        for (int[] e : edges){ g[e[0]].add(e[1]); g[e[1]].add(e[0]); }

        vis = new boolean[n]; System.out.print("BFS: "); bfs(0); // 0 1 2 3 4
        System.out.println();
        vis = new boolean[n]; System.out.print("DFS: "); dfs(0); // 0 1 3 2 4 (order may vary)
    }
}

```

Graph: 0-1-3-4 0-2-3

BFS from 0 : queue levels -> 0 | 1,2 | 3 | 4 visit: 0 1 2 3 4

DFS from 0 : go deep 0 -> 1 -> 3 -> 2 (dead end) back -> 4
visit: 0 1 3 2 4

4.4 Minimum Spanning Tree (MST) — Kruskal & Prim

DEFINITION: SPANNING TREE & MST

A spanning tree of a connected graph is a sub-graph with all V vertices and exactly V-1 edges (no cycle, still connected). The MINIMUM spanning tree is the spanning tree with the minimum total edge weight. Real use: laying optical cables / power lines between cities with minimum cost, network design, cluster analysis.

Graph:

```

  A--6--B--5--C
  |   4   \   \
  8   D    2   7
  |   /     |   /
  E--9--F    G--8

```

MST (total weight 19) with Kruskal's picking smallest edges first:
5(B-C), 2(C-G), 6(A-B), 4(B-D), 8(A-E), ... skip 9(F-E already)

Kruskal.java - sort edges + union-find (disjoint set)

```
import java.util.*;
class Kruskal {
    static int[] parent;
    static int find(int x){ if(parent[x]==x) return x;
        return parent[x] = find(parent[x]); } // path compression
    static void union(int a,int b){ parent[find(a)] = find(b); }
    static int mst(int n, List<int[]> edges){ // edge = {w,u,v}
        Collections.sort(edges, Comparator.comparingInt(e->e[0])); // sort by weight
        parent = new int[n];
        for (int i=0;i<n;i++) parent[i] = i;
        int cost = 0, taken = 0;
        for (int[] e : edges){
            int u=e[1], v=e[2];
            if (find(u) != find(v)) { union(u,v); cost += e[0]; taken++; }
            if (taken == n-1) break;
        }
        return cost;
    }
    public static void main(String[] a){
        List<int[]> edges = new ArrayList<>();
        Collections.addAll(edges, new int[]{6,0,1},{5,1,2},{4,1,3},
            new int[]{8,0,4},{2,2,3},{9,4,5});
        System.out.println("MST weight = " + mst(6, edges)); // 23
    }
}
```

Prim.java - grow one tree from a vertex with a priority queue

```
import java.util.*;
class Prim {
    static int mst(int n, ArrayList<int[]> g){ // g[u] = {v, w}
        boolean[] in = new boolean[n];
        int[] key = new int[n]; Arrays.fill(key, Integer.MAX_VALUE);
        PriorityQueue<int[]> pq = new PriorityQueue<>(
            Comparator.comparingInt(e->e[0])); // {weight, vertex}
        pq.add(new int[]{0, 0}); key[0] = 0;
        int cost = 0;
        while(!pq.isEmpty()){
            int[] top = pq.poll(); int u = top[1];
            if (in[u]) continue; // stale entry
            in[u] = true; cost += top[0];
            for (int[] e : g[u]){
                int v=e[0], w=e[1];
                if (!in[v] && w < key[v]) { key[v]=w; pq.add(new int[]{w,v}); }
            }
        }
        return cost;
    }
}
// Prim grows ONE tree from a vertex, greedily adding the cheapest EDGE
// that connects the tree to a new vertex. Uses a priority queue = O(E log V).
```

	Kruskal	Prim
Idea	Pick globally smallest edges, avoid cycle (union-find)	Grow a single tree, add cheapest connecting edge
Data structures	Sorted edge list + Disjoint-set	Adjacency list + Min-heap
Works on	Edge list (any representation)	Adjacency list / matrix
Complexity	$O(E \log E)$	$O(E \log V)$ / $O(V^2)$ on matrix
Real use	Network cable layout, cluster analysis	Designing road/electric grids, MST in maps

4.5 Dijkstra's Shortest Path Algorithm

DEFINITION: DIJKSTRA

Finds shortest path from one source to ALL vertices in a graph with NON-NEGATIVE edge weights. Repeatedly relaxes edges from the unvisited vertex with the smallest known distance. $O((V+E) \log V)$ with a priority queue.

Dijkstra.java - priority-queue based shortest path

```
import java.util.*;
public class Dijkstra {
    static int[] dist;
    static void run(int n, ArrayList<int[]>[] g, int src){
        dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;
        boolean[] done = new boolean[n];
        PriorityQueue<int[]> pq = new PriorityQueue<>(
            Comparator.comparingInt(e->e[0]));
        pq.add(new int[]{0, src});
        while(!pq.isEmpty()){
            int[] top = pq.poll();
            int u = top[1];
            if (done[u]) continue;      // stale copy
            done[u] = true;
            for (int[] e : g[u]) {
                int v=e[0], w=e[1];
                if (!done[v] && dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;    // RELAXATION
                    pq.add(new int[]{dist[v], v});
                }
            }
        }
    }
    public static void main(String[] a){
        int n = 5;
        ArrayList<int[]>[] g = new ArrayList[n];
        for (int i=0;i<n;i++) g[i] = new ArrayList<>();
        int[][] ed = {{0,1,4},{0,2,1},{2,3,2},{1,3,1},{3,4,6}};
        for (int[] e : ed){ g[e[0]].add(new int[]{e[1],e[2]});
                           g[e[1]].add(new int[]{e[0],e[2]}); }
        run(n, g, 0);
        System.out.println(Arrays.toString(dist)); // [0, 4, 1, 3, 9]
    }
}
```

Algorithm	Problem	Technique	Complexity	Big limitation
BFS	Shortest hops (unweighted)	Queue, level-order	$O(V+E)$	No weights
DFS	Reachability, components, cycles	Stack / recursion	$O(V+E)$	Not shortest-path
Kruskal	MST	Greedy + union-find	$O(E \log E)$	Needs sorting
Prim	MST	Greedy + heap	$O(E \log V)$	Vertex-centric growth
Dijkstra	Single-source shortest path	Greedy relaxation + heap	$O((V+E) \log V)$	Negative edges (use Bellman-Ford)

Real world of graphs: Google Maps & GPS routing (Dijkstra/A*), Facebook friend recommendation & communities (BFS), internet routers (OSPF uses Dijkstra), game path-finding (A* = Dijkstra+heuristic), e-commerce recommendation similarity graphs, deadlock detection (cycles in wait-for graph), dependency graphs of software packages (topological sort), network redundancy (MST).

Exercise: 1) Run Kruskal on the 3.4 tree example on paper and find MST weight. 2) Why does Dijkstra fail on negative weights? 3) Modify DFS to detect a cycle. 4) Draw BFS tree of the graph in 4.3.

UNIT 5 - SORTING, SEARCHING & HASHING

5.1 Sorting — Introduction & Terminology

DEFINITION: SORTING

Arranging elements of a list in ascending/descending order. Stable sort keeps the relative order of equal keys (important when sorting by 2 keys). In-place sort uses $O(1)$ extra memory. Total vs partial order, internal (RAM) vs external (disk) sorting.

Sort	Idea	Best Average Worst	Space	Stable
Bubble	Repeatedly swap adjacent out-of-order pairs	$O(n)$ $O(n^2)$ $O(n^2)$	$O(1)$	Yes
Selection	Pick min each pass, place at front	$O(n^2)$ $O(n^2)$ $O(n^2)$	$O(1)$	No
Insertion	Insert each element into sorted prefix	$O(n)$ $O(n^2)$ $O(n^2)$	$O(1)$	Yes
Shell	Insertion sort on shrinking gaps	$O(n \log n)$ * $O(n^2)$	$O(1)$	No
Merge	Divide, sort halves, merge	$O(n \log n)$ always	$O(n)$	Yes
Quick	Partition by pivot, recurse	$O(n \log n)$ $O(n \log n)$ $O(n^2)$	$O(\log n)$	No
Heap	Build max-heap, extract root	$O(n \log n)$ always	$O(1)$	No
Radix	Digit-wise (bucket) passes, non-comparison	$O(d*(n+k))$	$O(n+k)$	Yes

5.2 Bubble, Selection, Insertion Sort

SimpleSorts.java - three $O(n^2)$ classics

```

public class SimpleSorts {
    static void swap(int[] a, int i, int j){ int t=a[i];a[i]=a[j];a[j]=t; }

    static void bubble(int[] a){                //  $O(n^2)$  worst
        int n=a.length;
        for (int i=0;i<n-1;i++){
            boolean swapped=false;
            for (int j=0;j<n-1-i;j++){
                if (a[j]>a[j+1]){ swap(a,j,j+1); swapped=true; }
            }
            if(!swapped) break;                // already sorted ->  $O(n)$ 
        }
    }
    static void selection(int[] a){              //  $O(n^2)$  always
        int n=a.length;
        for (int i=0;i<n-1;i++){
            int m=i;                            // find minimum in rest
            for (int j=i+1;j<n;j++) if (a[j]<a[m]) m=j;
            swap(a,i,m);
        }
    }
    static void insert(int[] a){                 // best  $O(n)$  nearly sorted
        int n=a.length;
        for (int i=1;i<n;i++){
            int key=a[i], j=i-1;
            while (j>=0 && a[j]>key){ a[j+1]=a[j]; j--; }
            a[j+1]=key;
        }
    }
    public static void main(String[] s){
        int[] a={64,34,25,12,22,11,90};
        insert(a);
        System.out.println(java.util.Arrays.toString(a)); // [11,12,22,25,34,64,90]
    }
}

```

5.3 Shell Sort

DEFINITION: SHELL SORT

Improvement of insertion sort: first sort far-away elements using decreasing GAPS (e.g., $n/2$, $n/4$, ... , 1). Large values jump quickly to their side; final gap-1 pass is a cheap insertion sort.

ShellSort.java - main method

```

static void shell(int[] a){
    int n = a.length;
    for (int gap = n/2; gap > 0; gap /= 2){      // shrinking gaps
        for (int i = gap; i < n; i++){          // insertion sort with step 'gap'
            int key = a[i], j = i;
            while (j-gap >= 0 && a[j-gap] > key){ a[j] = a[j-gap]; j -= gap; }
            a[j] = key;
        }
    }
}

```

// trace: [9,8,7,6,5] gap=2 sorts pairs (9,7,5)->(9,7,6,8,5)... gap=1 finishes

5.4 Merge Sort & Quick Sort (divide and conquer)

DivideAndConquer.java - merge + quick sort

```

import java.util.Arrays;
public class DivideAndConquer {
    // ----- MERGE SORT : O(n log n), stable, O(n) space -----
    static void merge(int[] a, int l, int m, int r){
        int[] L = Arrays.copyOfRange(a, l, m+1);    // left copy
        int[] R = Arrays.copyOfRange(a, m+1, r+1);    // right copy
        int i=0, j=0, k=l;
        while (i<L.length && j<R.length)
            a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
        while (i<L.length) a[k++] = L[i++];
        while (j<R.length) a[k++] = R[j++];
    }
    static void mergeSort(int[] a, int l, int r){
        if (l >= r) return;                // single element = sorted
        int m = l + (r-l)/2;
        mergeSort(a, l, m); mergeSort(a, m+1, r); // divide
        merge(a, l, m, r);                 // conquer / combine
    }

    // ----- QUICK SORT : O(n log n) avg, in-place -----
    static int partition(int[] a, int lo, int hi){
        int pivot = a[hi];                // last element pivot
        int i = lo - 1;                   // boundary of smaller half
        for (int j = lo; j < hi; j++)
            if (a[j] < pivot) { i++; int t=a[i]; a[i]=a[j]; a[j]=t; }
        int t = a[i+1]; a[i+1]=a[hi]; a[hi]=t;    // pivot in position
        return i+1;
    }
    static void quickSort(int[] a, int lo, int hi){
        if (lo >= hi) return;
        int p = partition(a, lo, hi);
        quickSort(a, lo, p-1); quickSort(a, p+1, hi);
    }
    public static void main(String[] s){
        int[] a = {38,27,43,3,9,82,10};
        mergeSort(a, 0, a.length-1);
        System.out.println(Arrays.toString(a));    // [3,9,10,27,38,43,82]
        int[] b = {10,7,8,9,1,5};
        quickSort(b, 0, b.length-1);
        System.out.println(Arrays.toString(b));    // [1,5,7,8,9,10]
    }
}

```

MERGE sort recursion tree (n=8):

[8 elems]

/ \

[4] [4]

/ \ / \

[2][2] [2][2]

merge 2 halved lists takes O(n) per level, log n levels = O(n log n)

QUICK sort partition pass:

pivot=5

[1 2 0 3 |5| 8 9 7]

(divide until size 1, small |5| big

then MERGE back = combine) left -> [0 1 2 3]

right -> [7 8 9]

5.5 Heap Sort & Radix Sort

HeapRadix.java - heap sort & LSD radix sort

```

import java.util.Arrays;
public class HeapRadix {
    // ----- HEAP SORT : build max-heap, extract root n times -----
    static void heapify(int[] a, int n, int i){
        int big = i, l = 2*i+1, r = 2*i+2;
        if (l < n && a[l] > a[big]) big = l;
        if (r < n && a[r] > a[big]) big = r;
        if (big != i){ int t=a[i];a[i]=a[big];a[big]=t; heapify(a,n,big); }
    }
    static void heapSort(int[] a){
        int n = a.length;
        for (int i = n/2-1; i >= 0; i--) heapify(a, n, i); // build heap
        for (int i = n-1; i > 0; i--){ // extract
            int t=a[0]; a[0]=a[i]; a[i]=t;
            heapify(a, i, 0);
        }
    }
    // ----- RADIX SORT : counting sort per digit -----
    static void counting(int[] a, int exp){
        int[] out = new int[a.length];
        int[] cnt = new int[10];
        for (int x : a) cnt[(x/exp)%10]++;
        for (int i=1;i<10;i++) cnt[i]+=cnt[i-1];
        for (int i=a.length-1;i>=0;i--){
            out[--cnt[(a[i]/exp)%10]] = a[i];
            System.arraycopy(out,0,a,0,a.length);
        }
    }
    static void radix(int[] a){
        int m = Arrays.stream(a).max().getAsInt(); // find max
        for (int exp=1; m/exp>0; exp*=10) counting(a, exp); // LSD pass
    }
    public static void main(String[] s){
        int[] h = {12,11,13,5,6,7}; heapSort(h);
        System.out.println(Arrays.toString(h)); // [5,6,7,11,12,13]
        int[] r = {170,45,75,90,802,24,2,66}; radix(r);
        System.out.println(Arrays.toString(r)); // [2,24,45,66,75,90,170,802]
    }
}

```

RADIX (LSD) sorting 170,45,75,90,802,24,2,66 by digits:
 pass1 (units): 170,90,802,2,... stable buckets keep order
 pass2 (tens) : 802,02,24,45,66,170,75,90
 pass3 (hundreds): 2,24,45,66,75,90,170,802 => SORTED in $O(3*(n+10))$
 (Counting sort inside each pass is stable - critical for radix.)

5.6 Searching — Sequential, Binary, Comparison

SEQUENTIAL (LINEAR) SEARCH

Scan from first to last until key found. Works on UNSORTED data. Average $O(n)$, a sorted array does not help in worst case. Use for small lists.

BINARY SEARCH

Data must be SORTED. Repeatedly halve the search space: compare middle element; go left if key smaller, right if larger. $O(\log n)$. Divide-and-conquer; can be iterative or recursive.

Searching.java - linear vs binary

```

import java.util.Arrays;
public class Searching {
    static int linear(int[] a, int key){           // O(n)
        for (int i=0;i<a.length;i++) if (a[i]==key) return i;
        return -1;
    }
    static int binary(int[] a, int key){          // O(log n) - sorted only
        int lo=0, hi=a.length-1;
        while (lo <= hi){
            int mid = lo + (hi-lo)/2;
            if (a[mid]==key) return mid;
            if (a[mid] < key) lo = mid+1;
            else hi = mid-1;
        }
        return -1;
    }
    public static void main(String[] h){
        int[] a = {10,22,35,48,59,72};
        System.out.println(linear(a, 35)); // 2
        System.out.println(binary(a, 72)); // 5 (2 comparisons only)
        // Java built-ins: Arrays.binarySearch(a,key), Collections.binarySearch
    }
}

```

	Sequential search	Binary search
Precondition	None	Sorted data
Complexity	O(n) avg/worst, O(1) best	O(log n) always
Data structure	Array or linked list	Array with random access (not LL easily)
Comparisons for n=1,000,000	~500,000 (avg)	~20
Java built-in	(none needed)	Arrays.binarySearch

5.7 Hashing & Indexing

DEFINITION: HASHING

Maps a KEY directly to a memory address (index) using a HASH FUNCTION $h(\text{key})$. Search/insert/delete become $O(1)$ average, instead of $O(n)$. Collisions (two keys $\rightarrow$ same index) are resolved by chaining (linked list in the slot) or open addressing (linear/quadratic probing).

```

table size = 7,  h(key) = key % 7
insert 50  $\rightarrow$  1, 22  $\rightarrow$  1 (COLLISION)  $\rightarrow$  chain:

index: 0   1       2   3   4   5   6
      [ ][ 22]->[50] [ ] [31] [ ] [12] [30]
          chaining: same-slot keys in a linked list
open addressing: 22 goes to slot 2 if 1 is taken (probe next)
load factor  $\alpha = n/\text{tableSize}$ ; keep  $< 0.7$  for  $O(1)$  behaviour.

```

HashDemo.java - Java HashMap/HashSet (industry hashing)

```
import java.util.*;
public class HashDemo {
    public static void main(String[] a){
        // Java HashMap = chained hash table, O(1) average
        HashMap<String, Integer> marks = new HashMap<>();
        marks.put("Riya", 92); marks.put("Aman", 88);
        System.out.println(marks.get("Riya")); // 92 (hash of key)

        // HashSet = HashMap with only keys (used for dedup checks)
        HashSet<String> seen = new HashSet<>();
        seen.add("A"); seen.add("B"); seen.add("A");
        System.out.println(seen.size()); // 2

        // true hash map collision resolution in one line:
        // new HashMap<>(capacity, loadFactor) lets you tune buckets.
    }
}
```

INDEXING VS HASHING (DBMS)

Hashing: direct O(1) access to ONE row by exact key (used for hash join, memcache lookup). INDEXING (B+ tree): keeps keys sorted, supports RANGE queries ("marks between 50 and 70") and ordered scans; much larger data, disk-backed.

Real-world hashing: password digests (SHA-256), HashMap in every language, cache keys (Redis/Memcached), load balancing (consistent hashing of requests to servers), Bloom filters, spell-check dictionary lookups.

5.8 Case Study — Data Structures in OS & DBMS**A) Operating Systems**

Run queue Min-heap / priority queue schedules highest-priority process (Linux CFS uses a red-black tree).	Ready queue Linked list / queue of processes waiting for CPU.	Memory free blocks Free list (linked list) + buddy allocator (binary tree), LRU page cache (doubly-linked list + hash map).	File system B-tree family indexes directories + FAT table (linked list of clusters).
Deadlock detection Wait-for GRAPH; cycle = deadlock.	Interrupt stack Stack stores return addresses of nested interrupt handlers.	Printer spool Queue buffers print jobs (FIFO).	Keyboard buffer Circular queue stores typed keys for the CPU.

B) Database Management System (DBMS)

Table index B+ tree on primary key -> O(log n) lookups + range scans.	Hash index Exact-match lookups in O(1) (hash join, primary keys in mem).	Query execution plan Operator TREE (scan -> filter -> join -> group).	Buffer pool LRU cache: hash map + doubly linked list of pages.
Join algorithms Nested-loop (O(n*m)), merge join (sorted lists, merge), hash join (hashing).	Transaction undo log Stack/queue of actions to reverse on rollback.	Sort merge (ORDER BY) Merge sort for large external data.	B-tree vs LSM LSM-tree (Log-structured merge tree) for write-heavy workloads.

One-line summary: every real system is a composition of these five units - arrays/lists (data), stacks (undo/history), queues (scheduling), trees (indexes), graphs (relations), sorting/searching/hashing (processing).

DS	A Real Product Example	Which unit
Linked list	Photos app: next/previous photo, LRU cache	Unit 1
Stack	Editor Undo, compiler expression conversion	Unit 2
Queue / Heap	Printer spool, OS process priority queue	Unit 2

DS	A Real Product Example	Which unit
BST / B+ tree	MySQL primary key index	Unit 3
Graph + Dijkstra	Google Maps route, routers (OSPF)	Unit 4
Hash table	HashMap lookup, Redis cache	Unit 5

Exercises: 1) Trace merge sort on [5,9,2,7,1] with the tree diagram. 2) Which sort does Java's Arrays.sort use and why is it a hybrid? 3) Insert 41,58,32,100,75 into a hash table of size 10 with linear probing and draw the table. 4) Search key 90 in the BST of Unit 3 and count comparisons.